

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	P Karthik	Reg. No.:	192524222
Section / Batch:	CSE (AIDS)	Date:	23/07/2026

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I P Karthik(192524222) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:
P Karthik

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Debugging Selection Sort Code

The following Selection Sort code contains logical errors and produces incorrect output for some inputs. Carefully analyze the code, identify all errors in comparison and swapping logic, and explain their root causes. Apply systematic debugging to correct the algorithm step-by-step. Optimize the implementation by reducing unnecessary comparisons if possible. Analyze the corrected algorithm in terms of time complexity. Rewrite the final optimized code with proper structure and comments.

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        min_idx = i  
        for j in range(i+1, n):  
            if arr[j] > arr[min_idx]: # ERROR  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  
    return arr
```

Errors in the Code

- Comparison Error: `arr[j] > arr[min_idx]` finds the largest element instead of the smallest.
- Swapping Error: Swapping is performed inside the inner loop, causing incorrect intermediate results and unnecessary swaps.

Root Causes

- The comparison operator should be `<` to select the minimum element.
- Swapping should occur after the inner loop, once the minimum element has been found.

Corrected Steps

- Initialize `min_idx = i`.
- Compare all remaining elements to find the smallest.
- Swap only once after completing the inner loop.

- Skip the swap if `min_idx == i` to avoid unnecessary swaps.

Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Space Complexity: $O(1)$

Corrected Code

```
def selection_sort(arr):  
    n = len(arr)  
  
    for i in range(n - 1):  
        min_idx = i  
  
        # Find the smallest element  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
  
        # Swap only if needed  
        if min_idx != i:  
            arr[i], arr[min_idx] = arr[min_idx], arr[i]  
  
    return arr
```

Conclusion

The corrected Selection Sort correctly sorts the array in ascending order. Moving the swap outside the inner loop and avoiding unnecessary swaps improves efficiency while maintaining $O(n^2)$ time complexity.

Q2. Debugging Linear Search Code

The given Linear Search implementation fails to return correct results when the element is present and sometimes terminates incorrectly. Identify errors in loop conditions and return statements, and explain their causes. Debug the code to ensure correct functionality. Optimize the implementation for better readability and efficiency. Analyze its time complexity and justify the result.

Provide a clean and well-documented corrected version.

```
def linear_search(arr, key):  
for i in range(len(arr)):  
if arr[i] == key:  
return -1 # ERROR  
return i # ERROR
```

Errors in the Code

- Return Error 1: return -1 is executed when the element is found. It should return the index i.
- Return Error 2: return i is outside the if statement, causing the function to stop after the first iteration even if the element is not found.

Root Causes

- The return statements are placed incorrectly.
- The function should return -1 only after checking all elements.

Corrected Steps

- Traverse the array from beginning to end.
- If the key is found, return its index.
- If the loop finishes without finding the key, return -1.

Time Complexity

- Best Case: $O(1)$
- Average Case: $O(n)$
- Worst Case: $O(n)$
- Space Complexity: $O(1)$

Corrected Code

```
def linear_search(arr, key):  
    # Traverse the array  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i    # Return index if found  
  
    return -1    # Return -1 if not found
```

Conclusion

The corrected Linear Search correctly returns the index of the searched element or -1 if the element is not present. It is simple, efficient for small or unsorted datasets, and has a time complexity of $O(n)$.

Q3. Debugging Convex Hull Logic

The given brute force Convex Hull code incorrectly includes interior points as boundary points. Identify logical errors in orientation checking and explain their impact. Debug the algorithm to correctly identify hull edges. Optimize the implementation by reducing unnecessary checks. Analyze the time complexity of the corrected solution. Present a clean and structured version of the improved code.

```
def orientation(p, q, r):
```

```
    return (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1]) # SIGN ISSUE
```

```
# used incorrectly in hull condition
```

Debugging Convex Hull Logic

Errors in the Code

- The orientation formula uses the wrong sign, giving incorrect clockwise/counter-clockwise results.
- The hull condition uses the orientation result incorrectly, causing interior points to be included as hull edges.

Root Causes

- Incorrect orientation calculation leads to wrong side checking.
- A hull edge is valid only if all other points lie on the same side of the line.

Corrected Steps

- Use the correct orientation formula.
- Check all remaining points for each line pair.
- Accept the edge only if all points are on one side.
- Skip unnecessary checks once points are found on both sides.

Time Complexity

- Best Case: $O(n^3)$

- Average Case: $O(n^3)$
- Worst Case: $O(n^3)$
- Space Complexity: $O(1)$

Corrected Code

```
def orientation(p, q, r):
    # Returns positive, negative, or zero
    return (q[0] - p[0]) * (r[1] - p[1]) - (q[1] - p[1]) * (r[0] - p[0])

def convex_hull_bruteforce(points):
    n = len(points)
    hull = []

    for i in range(n):
        for j in range(i + 1, n):
            left = right = False

            for k in range(n):
                if k == i or k == j:
                    continue

                val = orientation(points[i], points[j], points[k])

                if val > 0:
                    left = True
                elif val < 0:
                    right = True

            # Stop checking if points exist on both sides
            if left and right:
                break

        if not (left and right):
            hull.append((points[i], points[j]))
```

```
return hull
```

Conclusion

The corrected algorithm identifies only the true convex hull edges by using the correct orientation test and avoiding unnecessary checks. Its time complexity remains $O(n^3)$, making it suitable for small datasets.

Q4. Debugging Binary Search Code

The following Binary Search implementation fails for certain inputs due to incorrect boundary updates. Identify all logical errors and explain their causes. Debug the code to handle all edge cases correctly. Optimize the implementation to avoid overflow in mid calculation. Analyze its performance in best, average, and worst cases. Rewrite the corrected code with proper documentation.

```
def binary_search(arr, key):  
    low, high = 0, len(arr)-1  
    while low < high:  
        mid = (low + high) // 2  
        if arr[mid] == key:  
            return mid  
        elif arr[mid] < key:  
            high = mid - 1 # ERROR  
        else:  
            low = mid + 1 # ERROR  
    return -1
```

Debugging Binary Search Code

Errors in the Code

- The loop condition `while low < high` may skip checking the last element. It should be `while low <= high`.
- When `arr[mid] < key`, the code incorrectly updates `high = mid - 1`. It should update `low = mid + 1`.

- When `arr[mid] > key`, the code incorrectly updates `low = mid + 1`. It should update `high = mid - 1`.

Root Causes

- Incorrect boundary updates cause the search to move in the wrong direction.
- The loop condition misses valid search cases when `low == high`.

Corrected Steps

- Use `while low <= high`.
- If `arr[mid] < key`, set `low = mid + 1`.
- If `arr[mid] > key`, set `high = mid - 1`.
- Calculate the middle index as `mid = low + (high - low) // 2` to avoid overflow.

Time Complexity

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$
- Space Complexity: $O(1)$

Corrected Code

```
def binary_search(arr, key):  
    low, high = 0, len(arr) - 1  
  
    while low <= high:  
        # Overflow-safe mid calculation  
        mid = low + (high - low) // 2  
  
        if arr[mid] == key:  
            return mid  
        elif arr[mid] < key:  
            low = mid + 1  
        else:  
            high = mid - 1
```

return -1

Conclusion

The corrected Binary Search handles all edge cases correctly by updating the search boundaries properly. It efficiently searches a sorted array with $O(\log n)$ time complexity and $O(1)$ space complexity.

Q6. Debugging Bubble Sort Code

The following Bubble Sort implementation does not correctly sort the array and performs unnecessary iterations. Identify logical errors in loop conditions and swapping mechanism, and explain their causes. Debug the code step-by-step to ensure correct sorting. Optimize the algorithm by reducing unnecessary passes using a flag. Analyze the time complexity of the corrected version. Rewrite the final optimized code with proper comments.

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(n-1):  
            if arr[j] < arr[j+1]: # ERROR  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
    return arr
```

Errors in the Code

- The condition `arr[j] < arr[j+1]` sorts the array in descending order. For ascending order, it should be `arr[j] > arr[j+1]`.
- The inner loop for `j in range(n-1)` performs unnecessary comparisons in every pass.
- The algorithm continues even when the array is already sorted.

Root Causes

- Incorrect comparison operator causes the wrong sorting order.
- The inner loop does not reduce after each pass.
- No flag is used to detect if the array is already sorted.

Corrected Steps

- Change the comparison to `arr[j] > arr[j+1]`.

- Reduce the inner loop to range(n-i-1).
- Use a swapped flag to stop early if no swaps occur.

Time Complexity

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Space Complexity: $O(1)$

Corrected Code

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        # Compare adjacent elements
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # Stop if no swaps occurred
        if not swapped:
            break

    return arr
```

Conclusion

The corrected Bubble Sort sorts the array correctly in ascending order. Using the swapped flag avoids unnecessary passes, improving the best-case time complexity to $O(n)$ while maintaining $O(n^2)$ in the average and worst cases.